BASICS OF AI ML AND DL

Artificial Intelligence (AI) is a technology that enables computers and machines to simulate human intelligence. This includes learning from data, understanding information, solving problems, making decisions, and sometimes even exhibiting creativity and autonomy.

Weak AI: Also known as "narrow AI," defines AI systems designed to perform a specific task or a set of tasks. Examples might include "smart" voice assistant apps, such as Amazon's Alexa, Apple's Siri, a social media chatbot or the autonomous vehicles promised by Tesla.

Strong AI Also known as "artificial general intelligence" (AGI) or "general AI," possess the ability to understand, learn and apply knowledge across a wide range of tasks at a level equal to or surpassing human intelligence.

Machine Learning (ML) is a subset of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed. In simple terms, ML allows machines to improve their performance automatically as they are exposed to more data.

Deep Learning is a subset of Machine Learning that uses multilayered neural networks, called deep neural networks, to model complex patterns and simulate human-like decision-making.

Generative AI, or Gen AI, refers to deep learning models that can create original content—such as text, images, audio, or video—based on a user's input. These models learn patterns from existing data to generate realistic and creative outputs.

Types of Machine Learning

1. Supervised Learning

- Definition: The model learns from labeled data, where both inputs and desired outputs are provided, to make predictions on new data.
- Example: Predicting house prices based on features like size and location.

2. Unsupervised Learning

- Definition: The model analyzes unlabeled data to identify hidden patterns, structures, or groupings without predefined outputs.
- Example: Customer segmentation based on purchasing behavior.

3. Semi-Supervised Learning

- Definition: The model uses a combination of labeled and unlabeled data to improve learning efficiency and accuracy.
- Example: Image recognition where only some images are labeled.

4. Reinforcement Learning

- Definition: The model learns by interacting with an environment, receiving rewards or penalties, and optimizing actions to achieve long-term goals.
- Example: Training a robot to walk or a system to play games like chess or Go.

Difference between Regression Problem and Classification Problem

1. Regression predicts a number, classification predicts a category
2. Regression uses Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 as its evaluation metrics and Classification uses Accuracy, Precision, Recall, F1-score as its evaluation metrics

NEURAL NETWORK

A neural network (also called an artificial neural network) is an adaptive system that learns by using interconnected nodes or neurons in a layered structure that resembles a human brain.

It consists of an input layer, one or more hidden layers, and an output layer. In each layer there are several nodes, or neurons, with each layer using the output of the previous layer as its input, so neurons interconnect the different layers.

Neural network's behaviour is defined by the way its individual elements are connected and by the strength, or weights of those connections. These weights are automatically adjusted during training according to a specified learning rule until the artificial neural network performs the desired task correctly.

A shallow neural network has only one hidden layer between the input and output. A deep neural network has two or more hidden layers between input and output.

NEURON'S LAYERS

Neurons are organized into multiple layers — an input layer, hidden layers, and an output layer.

1. Input Layer: This is where the network receives input from your dataset. It's often transformed into a suitable form for the network.

2. Hidden Layer(s): These are the layers in between the input and output layers. They perform transformations on the input data with the goal of learning features that are useful for the task at hand. The term "deep" in deep learning refers to having multiple hidden layers in the network.

3. Output Layer: This is the final layer. It provides the predictions or classifications that the network has been trained to produce.

PERCEPTRON

A perceptron is a type of artificial neuron or linear binary classifier that takes multiple input values, applies corresponding weights, adds a bias, and passes the result through an activation function to produce an output and is primarily used for binary classification tasks.

It was introduced by Frank Rosenblatt in 1958 and serves as the building block of more complex neural networks.

WEIGHTS AND BIAS

Weights determine how strongly each input feature influences the output of a neuron and its importance is influenced by its scaling.

Bias allows the activation function to be shifted left or right, helping the model better fit the data.

ACTIVATION FUNCTIONS

An Activation Function decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.

The primary role of the Activation Function is to transform the summed weighted input from the node into an output value to be fed to the next hidden layer or as output.

Activation functions introduces non-linearity in neural structure and without non-linearity, no matter how many layers the network has, it would behave just like a single layer network because summing these layers would give another linear function.

Non-linear activation functions allow the network to learn from the error, and adjust its weights and bias accordingly, so as to improve the model during training.

LOSS FUNCTION

Loss function is a mathematical function that measures the difference between the model's predicted output and the actual target value. It tells the model how wrong its predictions are. And it is also used to quantify errors.

VANISHING GRADIENT

The vanishing gradient problem in deep learning occurs during the training of deep neural networks when the gradients (used to update the network's weights via backpropagation) become extremely small—almost vanishing—as they propagate backward from the output layer to earlier layers. This causes the early layers of the network to update their weights very slowly or not at all, which significantly slows down learning or can even halt it completely.

This issue is especially prominent with activation functions like sigmoid and hyperbolic tangent (tanh), whose derivatives fall between 0 and 1 and approach zero when inputs saturate. As backpropagation multiplies these small derivatives layer by layer, the gradients shrink exponentially, making weight updates in earlier layers negligible. Consequently, the network struggles to learn from data in those layers, impeding effective training of deep networks.

How to recognise Vanishing Gradient Problem -

1. Calculate loss using Keras and if its consistent during epochs that means it is Vanishing Gradient Problem.

2. Draw the graphs between weights and epochs and if it is constant that means weight has not changed and hence vanishing gradient problem.

Several solutions help mitigate this problem -

1. Using activation functions like ReLU (Rectified Linear Unit) or its variants (leaky ReLU), which have gradients that do not vanish for positive inputs.

2. Implementing batch normalization to stabilize gradient flow.

3. Employing architectures with skip connections or residual networks (ResNets), allowing gradients to bypass certain layers.

4. Using gated recurrent units like LSTMs in sequence models.

5. Proper weight initialization and gradient clipping to maintain gradient magnitudes.

HOW IS DEEP LEARNING DIFFERENT AND BETTER FROM MACHINE LEARNING APPROACHES

Deep Learning differs from traditional Machine Learning by automatically learning hierarchical features, handling complex unstructured data, scaling with massive datasets, and achieving higher accuracy on tasks like computer vision, NLP, and speech recognition, whereas ML relies on manual feature engineering and works best for simpler, structured problems.

HISTORY OF AI

1950: Alan Turing asks "Can machines think?" and introduces the Turing Test to evaluate machine intelligence.

1956: John McCarthy coins “Artificial Intelligence,” and the first AI program, Logic Theorist, is created.

1967: Frank Rosenblatt builds the first neural network, the Mark 1 Perceptron, sparking early neural network research.

1980: Neural networks using backpropagation gain widespread use in AI applications.

1995: Russell and Norvig publish *Artificial Intelligence: A Modern Approach*, defining AI through rationality and thinking vs. acting.

1997: IBM’s Deep Blue defeats world chess champion Garry Kasparov.

2004: John McCarthy proposes a formal AI definition amid the rise of big data and cloud computing.

2011: IBM Watson wins *Jeopardy!* and data science emerges as a popular field.

2015: Baidu’s Minwa uses convolutional neural networks to outperform humans in image recognition.

2016: DeepMind’s AlphaGo defeats Go champion Lee Sedol, highlighting AI’s strategic capabilities.

2022: Large language models like ChatGPT revolutionize AI performance and enterprise applications.

2024: Multimodal AI models combine vision and language processing, and smaller efficient models advance alongside massive ones.

Internal Covariate Shift and Batch Normalization in Deep Learning

1. Internal Covariate Shift

Internal Covariate Shift (ICS) refers to the change in the distribution of inputs to a layer during training, caused by updates in the parameters of the previous layers. As each layer's parameters are updated, the output distribution changes, and the next layer must constantly adapt to new input distributions. This phenomenon can slow convergence and make optimization more difficult.

Key Characteristics:

- • Analogous to covariate shift in statistics, but occurs within the neural network.
- • Caused by weight updates during training that shift activation distributions.
- • Can lead to vanishing/exploding gradients in deep architectures.
- • Accumulates in deeper layers, increasing instability.

Impact on Training:

- • Slower convergence due to constantly changing distributions.
- • Requires smaller learning rates for stability.
- • Can lead to sub-optimal local minima.

2. Batch Normalization

Batch Normalization (BatchNorm) is a technique introduced by Sergey Ioffe and Christian Szegedy in 2015 to address ICS and improve the speed, performance, and stability of deep networks. It normalizes the input to each layer so that they have a mean close to 0 and variance close to 1, followed by a learnable scaling and shifting.

Step-by-Step Process:

1. 1. For each feature, calculate the mini-batch mean μ_B and variance σ^2_B.
2. 2. Normalize: subtract μ_B and divide by $\sqrt{(\sigma^2_B + \varepsilon)}$, where ε is a small constant for stability.
3. 3. Apply learnable scale (γ) and shift (β) parameters to allow the network to recover original representations if needed.

Mathematical Formulation:

- • $\mu_B = (1/m) \Sigma x_i$
- • $\sigma^2_B = (1/m) \Sigma (x_i - \mu_B)^2$
- • $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$
- • $y_i = \gamma \hat{x}_i + \beta$

Advantages of BatchNorm:

- • Reduces ICS and stabilizes learning.
- • Enables larger learning rates.
- • Provides regularization effect, reducing overfitting.
- • Helps maintain gradient flow, mitigating vanishing/exploding gradients.
- • Can make the network less sensitive to weight initialization.

Limitations:

- • Performance depends on batch size; small batches can cause noise in mean/variance estimates.
- • Adds computation and memory overhead.
- • Different behavior during training and inference (uses running estimates during inference).
- • May not be ideal for certain architectures like RNNs without modifications.

During Inference:

- • The mean and variance used are the moving averages computed during training.
- • Ensures deterministic outputs and stable predictions.

3. Relationship Between ICS and BatchNorm

BatchNorm was initially proposed as a direct solution to Internal Covariate Shift by keeping intermediate feature distributions stable. However, later research suggested its benefits also stem from smoothing the optimization landscape and improving gradient conditioning. While BatchNorm reduces ICS, its primary advantage may be enabling better training dynamics overall.

4. Variants and Alternatives

- • Layer Normalization – Normalizes across features for each sample, useful in RNNs.
- • Instance Normalization – Normalizes each channel separately, common in style transfer.
- • Group Normalization – Normalizes over groups of channels, works well for small batches.
- • Weight Normalization – Normalizes weights rather than activations.

5. Practical Tips

- • Place BatchNorm before the activation function in most cases (e.g., before ReLU).
- • Adjust momentum parameter in frameworks for better moving average estimates.
- • For very small batches, consider GroupNorm or LayerNorm instead.
- • Remember that BatchNorm parameters γ and β are trainable and can adapt.

Layer Normalization in Deep Learning

Layer Normalization (LayerNorm) is a normalization technique used in deep learning to stabilize and accelerate training by normalizing the activations across the features for each training example. Unlike Batch Normalization, which normalizes across the batch, LayerNorm operates independently for each sample, making it effective in settings with small batch sizes or sequential data.

1. How Layer Normalization Works

Given an input vector for a single training example, LayerNorm:

1. 1. Computes the mean and variance across all features of that example.
2. 2. Normalizes each feature by subtracting the mean and dividing by the standard deviation.
3. 3. Applies learnable scale (γ) and shift (β) parameters to allow the network to adapt.

2. Mathematical Formulation

Given an input vector $x = [x_1, x_2, \ldots, x_H]$ with H features:

- • Mean: $\mu = (1/H) \Sigma x_j$
- • Variance: $\sigma^2 = (1/H) \Sigma (x_j - \mu)^2$
- • Normalization: $\hat{x}_j = (x_j - \mu) / \sqrt{(\sigma^2 + \varepsilon)}$
- • Scaling & Shifting: $y_j = \gamma \hat{x}_j + \beta$

3. Advantages of LayerNorm

- • Works well with small batch sizes, even batch size = 1.
- • Effective in recurrent neural networks (RNNs) where batch statistics vary across time steps.
- • Invariant to batch size, providing consistent performance.
- • Reduces training instability and accelerates convergence.

4. Limitations of LayerNorm

- • Slightly more computation per sample compared to BatchNorm.
- • May be less effective than BatchNorm for large convolutional networks with large batch sizes.

5. Applications of LayerNorm

- • Transformers (used in attention layers for NLP and vision models).
- • RNNs, LSTMs, and GRUs to stabilize training across time steps.
- • Scenarios with small batch sizes where BatchNorm fails.

6. Comparison with Batch Normalization

- • BatchNorm normalizes across the batch dimension, LayerNorm normalizes across features for each sample.
- • BatchNorm requires consistent batch statistics, LayerNorm does not.
- • BatchNorm's performance can degrade for small batches; LayerNorm is unaffected.

Gradient Descent Optimizers in Deep Learning - Detailed Explanation

Batch Gradient Descent

Explanation: Batch Gradient Descent computes the gradient of the cost function with respect to the parameters for the entire dataset at each iteration. It guarantees convergence to the global minimum for convex functions and to a local minimum for non-convex functions. However, it can be slow for large datasets, and each update requires going through all training samples. It is best suited for small datasets where computational resources are not a constraint.

Formula: $\theta = \theta - \eta \nabla J(\theta)$

Explained Formula: Here, θ represents the model parameters (weights), η is the learning rate controlling the step size, and $\nabla J(\theta)$ is the gradient of the cost function with respect to θ. The update subtracts a fraction of the gradient from the current parameters to move towards the minimum.

When to Use: Use when datasets are small and you want stable, precise convergence.

Pros: Stable convergence; theoretically exact for convex problems.

Cons: Slow for large datasets; high memory requirements.

Stochastic Gradient Descent (SGD)

Explanation: SGD updates the parameters for each training example individually, making updates noisy but fast. The noise can help escape local minima, making it useful for non-convex optimization. However, due to the noise, the path to convergence can be irregular, requiring learning rate schedules or decay to stabilize the final solution.

Formula: $\theta = \theta - \eta \nabla J(\theta; x_i, y_i)$

Explained Formula: θ is updated using the gradient calculated from a single data point (x_i, y_i). This results in faster updates per iteration compared to batch gradient descent, but with more variance in the updates.

When to Use: Use when datasets are too large for batch processing.

Pros: Fast iterations; can escape local minima; low memory usage.

Cons: Noisy convergence; requires careful learning rate tuning.

Mini-Batch Gradient Descent

Explanation: Mini-Batch Gradient Descent splits the training dataset into small batches and updates the model parameters for each batch. This combines the advantages of both batch

and stochastic gradient descent by reducing variance in updates while allowing vectorized computation for efficiency. It is the most commonly used optimizer in deep learning.

Formula: $\theta = \theta - \eta \nabla J(\theta; B)$

Explained Formula: Here, B is a batch of data points. The gradient is computed over this small subset of the dataset, making the update faster than batch gradient descent and less noisy than SGD.

When to Use: Default choice for deep learning on large datasets.

Pros: Efficient computation; more stable than SGD.

Cons: Requires tuning of batch size; still some variance in updates.

Momentum

Explanation: Momentum helps accelerate gradient descent by accumulating a moving average of past gradients in the direction of consistent improvement. It reduces oscillations, especially in scenarios where the gradient direction changes frequently (e.g., narrow valleys).

Formula: $v_t = \beta v_{t-1} + \eta \nabla J(\theta)$; $\theta = \theta - v_t$

Explained Formula: v_t is the velocity term, β is the momentum coefficient ($0 < \beta < 1$), η is the learning rate, and $\nabla J(\theta)$ is the gradient. The velocity smooths updates, allowing faster movement in consistent directions.

When to Use: When gradients oscillate; to accelerate convergence.

Pros: Speeds up convergence; reduces oscillations.

Cons: May overshoot minima; requires tuning β.

Nesterov Accelerated Gradient (NAG)

Explanation: NAG improves on momentum by first making a lookahead step using the current velocity, then computing the gradient at that approximate future position. This provides more accurate gradient estimates, leading to better convergence.

Formula: $v_t = \beta v_{t-1} + \eta \nabla J(\theta - \beta v_{t-1})$; $\theta = \theta - v_t$

Explained Formula: First, a lookahead position $\theta - \beta v_{t-1}$ is computed, and the gradient is calculated at this point. Then, velocity and parameters are updated accordingly.

When to Use: When momentum is helpful but needs more precise updates.

Pros: More accurate updates; faster convergence.

Cons: Slightly more computationally expensive per iteration.

Adagrad

Explanation: Adagrad adapts the learning rate for each parameter individually based on the history of gradients. It performs larger updates for parameters with infrequent updates and smaller updates for parameters with frequent updates, making it ideal for sparse data.

Formula: $\theta = \theta - \eta / (\sqrt{G_t} + \varepsilon) \nabla J(\theta)$

Explained Formula: G_t is the sum of the squares of past gradients for each parameter. The learning rate η is divided by the root of G_t plus a small constant ε to avoid division by zero.

When to Use: When working with sparse data like text features.

Pros: No need to tune learning rate manually; works well with sparse features.

Cons: Learning rate decreases over time, possibly becoming too small.

RMSprop

Explanation: RMSprop maintains an exponentially decaying average of squared gradients and uses it to normalize updates. It is designed to fix Adagrad's diminishing learning rate issue, making it effective for non-stationary objectives.

Formula: $E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g^2_t$; $\theta = \theta - \eta / (\sqrt{E[g^2]_t} + \varepsilon) g_t$

Explained Formula: The denominator contains an exponentially weighted moving average of squared gradients, which helps maintain a consistent learning rate.

When to Use: When training RNNs or dealing with non-stationary problems.

Pros: Prevents diminishing learning rate; effective in practice.

Cons: Requires tuning β and η carefully.

Adam

Explanation: Adam combines the ideas of momentum and RMSprop. It maintains exponentially decaying averages of past gradients and squared gradients, applies bias correction, and updates parameters adaptively. It is widely used in deep learning.

Formula: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$; $v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2_t$; $\theta = \theta - \eta (\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon))$

Explained Formula: m_t is the first moment (mean of gradients), v_t is the second moment (mean of squared gradients), and $\hat{m}_t$, $\hat{v}_t$ are bias-corrected estimates. The parameters are updated using these adjusted moments.

When to Use: Default choice for most deep learning problems.

Pros: Works well without much tuning; fast convergence.

Cons: May overfit; can be sensitive to hyperparameters.

AdamW

Explanation: AdamW is a variant of Adam that decouples weight decay from the gradient update step. This improves generalization and has become the default choice in many transformer models.

Formula: $\theta = \theta - \eta (\hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda\theta)$

Explained Formula: Adam updates are computed normally, and a separate weight decay term $\lambda\theta$ is applied to the parameters to regularize them.

When to Use: When using Adam but needing better regularization.

Pros: Improved generalization; better than L2 regularization with Adam.

Cons: Requires tuning λ carefully.

What is generalization

In supervised learning, the main goal is to use training data to build a model that will be able to make accurate predictions based on new, unseen data, which has the same characteristics as the initial training set. This is known as generalization.

Generalization error

To train a machine learning model, you split the dataset into 3 sets: training, validation, and testing. we train your models using the training data, then we compare and tune them using the evaluation results on the validation set, and in the end, evaluate the performance of your best model on the testing set. The error rate on new cases is called the generalization error (or out-of-sample error)

A model's generalization error (also known as a prediction error) can be expressed as the sum of three very different errors: Bias error, variance error, and irreducible error.

Note: The irreducible error occurs due to the noisiness of the data itself. The only way to reduce this part of the error is to clean up the data (e.g., fix the data sources, such as broken sensors, or detect and remove outliers).

Bias error

Characteristics of a high-bias model include:

- Failure to capture proper data trends
- Potential towards underfitting
- More generalized/overly simplified
- High error rate

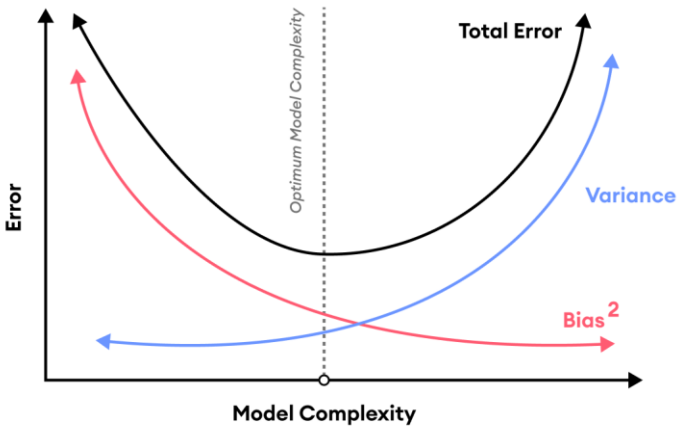
Variance error

A high-variance model typically has the following qualities:

- Noise in the data set
- Potential towards overfitting
- Complex models

Bias-variance tradeoff

Bias/variance in machine learning relates to the problem of simultaneously minimizing two error sources (bias error and variance error).



In order to find a balance between underfitting and overfitting (the best model possible), you need to find a model which minimizes the total error.

Total Error = Bias²+ Variance + Irreducible Error

What is underfitting

Underfitting occurs when a model is not able to make accurate predictions based on training data and hence, doesn't have the capacity to generalize well on new data.

Machine learning models with underfitting tend to have poor performance both in training and testing sets (like the child who learned only addition and was not able to solve problems related to other basic arithmetic operations both from his math problem book and during the math exam).

Underfitting models usually have high bias and low variance.

What is overfitting

A model is considered overfitting when it does extremely well on training data but fails to perform on the same level on the validation data (like the child who memorized every math problem in the problem book and would struggle when facing problems from anywhere else).

Models that are overfitting usually have low bias and high variance.

How to detect underfitting

1) **Training and test loss:** If the model is underfitting, the loss for both training and validation will be considerably high. In other words, for an underfitting dataset, the training and the validation error will be high.

2) **Oversimplistic prediction graph:** If a graph with the data points and the fitted curve is plotted, and the classifier curve is oversimplistic, then, most probably, your model is underfitting. In those cases, a more complex model should be tried out.

How to avoid underfitting

There are several things you can do to prevent underfitting in AI and machine learning models:

1) **Train a more complex model** – Lack of model complexity in terms of data characteristics is the main reason behind underfitting models. For example, you may have data with upwards of 100000 rows and more than 30 parameters. If you train data with the Random Forest model and set max depth (max depth determines the maximum depth of the tree) to a small number (for example, 2), your model will definitely be underfitting. Training a more complex model (in this respect, a model with a higher value of max depth) will help us solve the problem of underfitting.

2) **More time for training** - Early training termination may cause underfitting. As a machine learning engineer, you can increase the number of epochs or increase the duration of training to get better results.

3) **Eliminate noise from data** – Another cause of underfitting is the existence of outliers and incorrect values in the dataset. Data cleaning techniques can help deal with this problem.

4) **Adjust regularization parameters** - the regularization coefficient can cause both overfitting and underfitting models.

How to detect overfitting

Some of the techniques you can use to detect overfitting are as follows:

1) Use a resampling technique to estimate model accuracy. The most popular resampling technique is k-fold cross-validation. It allows you to train and test your model k-times on different subsets of training data and build up an estimate of the performance of a machine learning model on unseen data. The drawback here is that it is time-consuming and cannot be applied to complex models, such as deep neural networks.

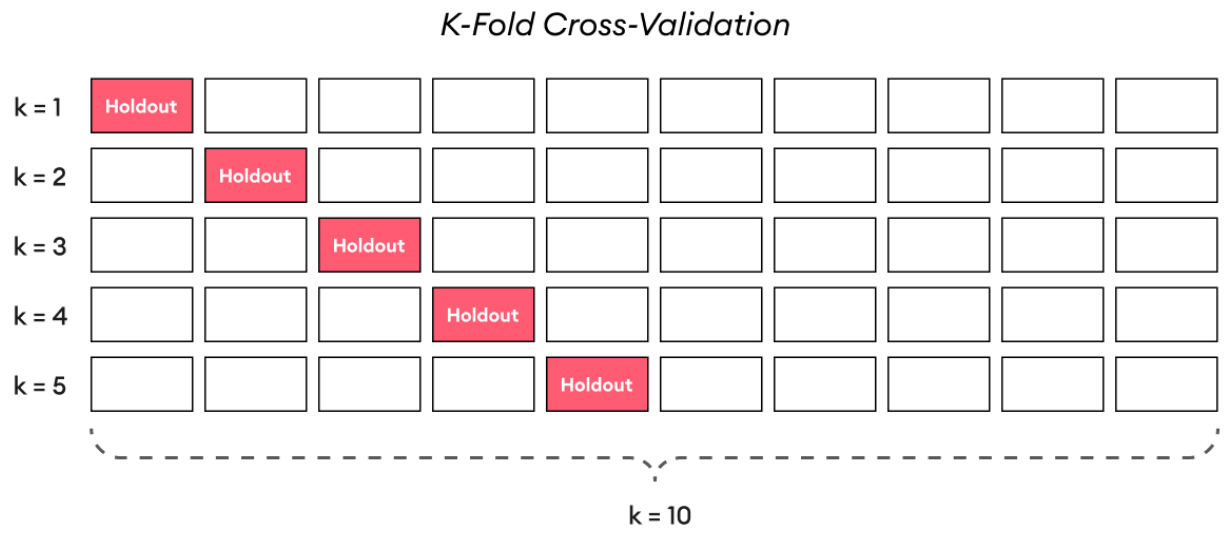


Figure 6. K-fold cross-validation: Image source

2) Hold back a validation set. Once a model is trained on the training set, you can evaluate it on the validation dataset, then compare the accuracy of the model in the training dataset and the validation dataset. A significant variance in these two results allows assuming that you have an overfitted model.

3) Another way to detect overfitting is by starting with a simplistic model that will serve as a benchmark. With this approach, if you try more complex algorithms, you will have a general understanding of whether the additional complexity for the model is worthwhile, if at all. This principle is known as *Occam's razor* test. This principle suggests that with all else being equal, simpler solutions to problems are preferred over more complex ones (if your model is not getting significantly better after using a much more complex model, it is preferable to use a simpler model).

How to prevent overfitting

Some of those methods are listed below.

1) **Adding more data** – Most of the time, adding more data can help machine learning models detect the "true" pattern of the model, generalize better, and prevent overfitting. However, this is not always the case, as adding more data that is inaccurate or has many missing values can lead to even worse results.

2) **Early stopping** – In iterative algorithms, it is possible to measure how the model iteration performance. Up until a certain number of iterations, new iterations improve the model. After that point, however, the model's ability to generalize can deteriorate as it begins to overfit the training data. Early stopping refers to stopping the training process before the learner passes that point.

3) **Data augmentation** – In machine learning, data augmentation techniques increase the amount of data by slightly changing previously existing data and adding new data points or by producing synthetic data from a previously existing dataset.

4) **Remove features** – You can remove irrelevant aspects from data to improve the model. Many characteristics in a dataset may not contribute much to prediction. Removing non-essential characteristics can enhance accuracy and decrease overfitting.

5) **Regularization** – Regularization refers to a variety of techniques to push your model to be simpler. The approach you choose will be determined by the model you are training. For example, you can add a penalty parameter for a regression (L1 and L2 regularization), prune a decision tree or use dropout on a neural network.

6) **Ensembling** – Ensembling methods merge predictions from numerous different models. These methods not only deal with overfitting but also assist in solving complex machine learning problems (like combining pictures taken from different angles into the overall view of the surroundings). The most popular ensembling methods are boosting and bagging.

- **Boosting** – In boosting method, you train a large number of weak learners (constrained models) in sequence, and each sequence learns from the mistakes of the previous sequence. Then you combine all weak learners into a single strong learner.

- **Bagging** is another technique to reduce overfitting. It trains a large number of strong learners (unconstrained models) and then combines them all in order to optimize their predictions.